Software Test Plan (STP) — Electronic Store Inventory Management System

Project: Electronic Store Inventory Management System | **Version:** 1.0 | **Authors:** Karnati Uday Keerthan Reddy, K B Balavideshna, Kolluri Sruthi | **Status:** Approved

1. Introduction

Purpose: This document defines the comprehensive Software Test Plan (STP) for the Electronic Store Inventory Management System v1.0. It specifies the testing objectives, scope, test items, features to be tested and excluded, test strategy, security validation, test environment, schedule phases, deliverables, roles, risks, and full traceability across the 18 formal test cases (UT_01–UT_02, IT_01–IT_12, ST_01–ST_04) and the 373 automated Pytest tests.
Scope: Testing encompasses unit, integration, route/RBAC, and system-level validation of user authentication, product catalog CRUD and search/filtering, multi-item POS sales checkout and invoice generation, customer returns and refunds, low-stock threshold alerts, supplier management, purchase order creation and full/partial delivery receipt, selectable date-range reporting (sales, stock, revenue) with CSV/PDF export, audit logging, and SQLite database backups.
References: (1) Software Requirements Specification (docs/SRS_Document.pdf, Documents/srs_table.pdf, Documents/security_requirements_specifications.pdf); (2) Software Architecture and Design Specification (docs/SAD_Document.pdf); (3) Unit, System Validation, Code Coverage, and Branch Coverage Reports (docs/).
Definitions: STP (Software Test Plan), SRS (Software Requirements Specification), RTM (Requirements Traceability Matrix), UT (Unit Test), IT (Integration Test), ST (System Test), RBAC (Role-Based Access Control), POS (Point of Sale), PO (Purchase Order), SKU (Stock Keeping Unit).

2. Test Items

- **Authentication & User Management Module:** app/presentation/routes/auth.py, app/presentation/authz.py, app/business/security.py, app/business/password_reset_email.py
- **Product Catalog, Search/Filter & Low-Stock Alert Module:** app/presentation/routes/main.py, app/data/models.py (Product)
- **Sales, POS Billing & Invoice Module:** app/business/sales.py (create_sale), app/presentation/routes/main.py (/sales, /sales/<id>/invoice)
- **Customer Returns, Refunds & Inventory Restock Module:** app/business/sales.py (process_return, restock_product, restock_inventory)
- **Supplier Management Module:** app/presentation/routes/supplier.py, app/data/models.py (Supplier)
- **Purchase Order & Partial Delivery Module:** app/business/purchase_order.py, app/presentation/routes/purchase_order.py
- **Analytical Reporting & CSV/PDF Export Module:** app/presentation/routes/main.py (/reports/sales, /reports/stock, /reports/revenue), app/business/report_exporter.py
- **Audit Trail & Database Backup Module:** app/data/models.py (AuditLog), scripts/backup_database.py

3. Features to be Tested

SRS Req ID	Feature Description	Verification Level	Planned Test References
FR-001	Add, update, and delete product records with unique SKU validation	Integration & Route	IT_03, IT_04, IT_05, test_products.py
FR-002	Automatic stock quantity update on sale, return, or restock	Unit & Integration	UT_01, IT_07, IT_08, IT_11, test_sales.py
FR-003	Automatic low-stock alert generation when stock falls below reorder threshold	Unit & System	ST_01, test_products.py
FR-004	Staff POS sale transaction creation and itemized invoice generation (5% tax)	Unit & Integration	UT_01, UT_02, IT_07, test_sales.py
FR-005	User authentication and role-based access restriction (Admin, Manager, Staff)	Integration & System	IT_01, IT_02, IT_09, ST_03, test_auth.py

SRS Req ID	Feature Description	Verification Level	Planned Test References
FR-006	Admin supplier creation, editing, and product-to-supplier linking	Integration	IT_09, test_suppliers.py
FR-007	Manager purchase order creation (Pending) and full receipt stock update (Received)	Integration	IT_08, test_purchase_orders.py
FR-008	Product search and filtering by name, SKU, category, or stock status	Integration	test_products.py, test_main_routes_coverage.py
FR-009	Selectable date-range Sales, Stock Movement, and Revenue report generation	System	ST_02, ST_04, test_sales_report.py, test_stock_report.py, test_revenue_report.py
FR-010	Audit logging of stock edits, deletions, and price changes with user ID and timestamp	Integration	IT_06, test_products.py
FR-011	Customer return and refund processing with automatic stock restoration	Integration	IT_11, test_sales.py
FR-012	Signed token-based password reset workflow for locked-out users	Integration	test_auth.py, test_password_reset_email.py
FR-013	Partial purchase order delivery recording and direct PO creation from low-stock alerts	Integration	test_purchase_orders.py
FR-014	CSV and PDF export of generated sales and stock movement reports	Integration	test_report_exports.py
FR-015	Admin user account creation, editing, deactivation, and reactivation	Integration	test_auth.py, test_auth_routes_coverage.py
NFR-003, NFR-005, NFR-009, NFR-010, NFR-012	Salted bcrypt hashing, atomic transaction rollback, daily SQLite backup integrity, 6-month audit retention, and clear validation messages	Unit & Integration	UT_02, IT_06, IT_10, IT_12, test_security.py, test_backup_database.py

4. Features Not to be Tested

- **NFR-002 (99% Production Store Uptime):** Requires continuous multi-week production hosting telemetry outside local/CI test execution.
- **NFR-004 (15-Minute New Staff Training Time):** Requires a formal human usability study with timed onboarding participants.
- **NFR-006 (50 Concurrent Users & 10,000 SKUs Load Test):** Multi-user stress/load benchmarking (e.g., Locust/JMeter) against a production WSGI cluster is outside the functional test suite scope.
- **NFR-011 / SEC-008 (Live HTTPS/TLS 1.2+ Network Capture):** Flask's in-process test_client() operates over local WSGI loopback without an external TLS reverse proxy.
- **Live External SMTP Delivery & Hardware Peripherals:** Outbound reset emails are verified via mock interception of send_reset_email(), and physical receipt printers/barcode scanners are external OS devices.

5. Test Approach / Strategy

Testing Levels:
• **Unit Testing (UT_01–UT_02 & Isolated Pytest Tests):** Direct invocation of domain functions and model properties (hash_password, verify_password, create_sale, Product.is_below_reorder_threshold, Supplier.to_dict) verifying pure logic and boundary conditions.
• **Integration Testing (IT_01–IT_12 & Route/DB Pytest Modules):** Verification of multi-layer interactions across Flask Blueprints, Business Layer services, and SQLAlchemy/SQLite persistence, including atomic multi-item rollbacks and audit log creation.
• **System Testing (ST_01–ST_04):** End-to-end workflow verification across low-stock alerting, date-range sales and stock movement reporting, and multi-role RBAC enforcement.
• **Acceptance Testing (UAT):** User-reported manual verification of all 18 formal test cases alongside automated regression execution.
Testing Types: Functional testing, boundary-value and negative input testing (@pytest.mark.parametrize), fault-injection rollback testing (monkeypatch on db.session.commit), security/RBAC validation, and statement/branch coverage measurement.

Entry Criteria: Application factory (create_app) initializes cleanly, SQLite test schema builds without error, and seeded role fixtures (conftest.py) are available.
Exit Criteria: 100% of the 18 planned formal test cases and 100% of the 373 automated Pytest tests pass with 0 failures/errors, ≥ 90% line and branch coverage achieved (actual: 97.70% line, 91.50% branch), and all high-priority functional requirements verified.

5.1 Security Validation

- **Password Hashing & Boundary Validation:** Verify that passwords are stored exclusively as salted bcrypt hashes ($2 prefix), unique salts are generated per user, and inputs exceeding bcrypt's 72-byte UTF-8 limit are deterministically rejected (test_security.py).
- **Server-Side Role-Based Access Control (RBAC):** Verify that anonymous users are redirected to /login (HTTP 302) and authenticated Staff or Manager accounts attempting unauthorized routes (/suppliers, /users, /purchase-orders, /reports/*) receive HTTP 403 Forbidden (IT_09, ST_03).
- **Session & Account Deactivation Controls:** Verify that deactivated users cannot authenticate, self-deactivation by an active Admin is blocked, and logout invalidates the active session.
- **Password Recovery Token Security:** Verify that password reset tokens are cryptographically signed via itsdangerous, expire after 30 minutes, reject tampered signatures, and return identical public responses for known and unknown emails (SEC-011).
- **Audit Trail Non-Repudiation & Atomic Rollback:** Verify that AuditLog records capture the server-authenticated current_user.id (ignoring spoofed actor parameters) and that mid-transaction failures leave zero partial stock or invoice changes (IT_06, IT_10).

6. Test Environment

Environment Category	Configuration / Specification
Operating System	macOS and Windows (both were used)
Programming Language	Python 3.14.6 (Repository virtual environment ./venv)
Application & Web Framework	Electronic Store Inventory Management System (Flask 3.1.3, Werkzeug 3.1.9, Flask-Login 0.6.3, Jinja2 3.1.6)
Database & ORM	SQLite 3 (isolated in-memory sqlite:///:memory: and temporary file DBs via tmp_path) with Flask-SQLAlchemy 3.1.1 / SQLAlchemy 2.1.6
Testing & Coverage Tools	Pytest 9.1.1, Flask Test Client (app.test_client()), pytest-cov 7.1.0, coverage.py 7.17.2, unittest.mock, Pytest monkeypatch
Test Data & Fixtures	Shared fixtures in tests/conftest.py seeding role accounts (test-admin, test-manager, test-staff), sample electronics SKUs, suppliers, purchase orders, and sales records

7. Test Schedule

Phase	Testing Milestone	Deliverable / Outcome	Status
Phase 1	Test Plan & Formal Test Case Design	18 formal test cases (UT_01–UT_02, IT_01–IT_12, ST_01–ST_04) mapped to SRS requirements	Completed
Phase 2	Test Environment & Fixture Setup	Isolated SQLite test configuration and role-based fixtures in tests/conftest.py	Completed
Phase 3	Automated Unit, Integration & Route Test Implementation	20 Pytest test_*.py modules comprising 373 automated test cases	Completed
Phase 4	Manual & Automated Execution + Coverage Analysis	373/373 automated tests passed; 18/18 manual test cases passed; 97.70% line & 91.50% branch coverage	Completed
Phase 5	Final Test Reporting & Sign-Off	Unit Test, System Validation, Code Coverage, Branch Coverage, and STP documents finalized	Completed

8. Test Deliverables

- **Software Test Plan (STP):** docs/Test_Plan_Document.pdf (this document) & Documents/test_plan.pdf
- **Automated Test Suite:** 20 test_*.py modules and tests/conftest.py in tests/ (373 automated tests)
- **Unit Test Report:** docs/unit_test_report.pdf (373 passed, 0 failed across 20 test modules)
- **System Validation Report:** docs/system_validation_report.pdf (18 formal test cases & 24 SRS requirements traceability)
- **Code Coverage Report:** docs/code_coverage_report.pdf (1,950 / 1,996 statements covered — 97.70%)
- **Branch Coverage Report:** docs/branch_coverage_report.pdf (635 / 694 branches covered — 91.50%)

9. Roles and Responsibilities

Role	Assigned Team / Member	Responsibilities
QA & Test Lead	Karnati Uday Keerthan Reddy, K B Balavideshna, Kolluri Sruthi	Prepare Software Test Plan, coordinate manual and automated test execution, and verify coverage targets.
Test Engineers	Karnati Uday Keerthan Reddy, K B Balavideshna, Kolluri Sruthi	Author and execute the 18 formal test cases and 373 automated Pytest tests across 20 test modules.
Backend & UI Developers	Karnati Uday Keerthan Reddy, K B Balavideshna, Kolluri Sruthi	Implement layered application modules, maintain test fixtures, and resolve any identified defects.
Course Evaluator / Product Owner	Shravya S Madhusudan	Review test deliverables, requirement traceability, and approve final system readiness.

10. Risks and Mitigation

Testing Risk	Mitigation Strategy
Shared database state causing flaky tests across modules	Isolate every test function using fresh in-memory SQLite databases (db.create_all() / db.drop_all()) in tests/conftest.py.

Testing Risk	Mitigation Strategy
External SMTP dependency failing during password reset tests	Mock outbound email dispatch (unittest.mock.patch on send_reset_email) while testing SMTP client logic separately.
Unverified error-recovery and rollback branches	Use Pytest monkeypatch to inject simulated SQLAlchemy commit/flush exceptions and verify atomic rollbacks.
Cross-platform path or OS differences	Execute and validate the test suite across both macOS and Windows environments using standard pathlib/os abstractions.

11. Assumptions & Dependencies

• The local Python virtual environment (./venv) contains all required packages pinned in requirements.txt.
• SQLite 3 provides deterministic ACID transaction semantics representative of the store's relational database.
• Flask's test_client() accurately models HTTP request routing, form encoding, session cookies, and status codes.

12. Suspension & Resumption Criteria

Suspension Criteria: Testing is suspended if application factory initialization (create_app) or database schema creation fails, blocking > 20% of test cases, or if a critical defect corrupts transaction rollback integrity.
Resumption Criteria: Testing resumes once the blocking defect is resolved, schema initialization succeeds, and smoke tests (tests/test_app.py) pass.

13. Test Case Management & Traceability

13.1 Requirement-to-Test-Case Traceability Summary

• **FR-001 (Product CRUD & Unique SKU):** IT_03, IT_04, IT_05, tests/test_products.py
• **FR-002 (Automatic Stock Updates):** UT_01, IT_07, IT_08, IT_11, tests/test_sales.py
• **FR-003 (Low-Stock Threshold Alerts):** ST_01, tests/test_products.py
• **FR-004 (Sale Transaction & Invoice Generation):** UT_01, UT_02, IT_07, tests/test_sales.py
• **FR-005 & NFR-003 (Authentication, bcrypt & RBAC):** IT_01, IT_02, IT_09, ST_03, tests/test_auth.py, tests/test_security.py
• **FR-006 (Supplier Management):** IT_09, tests/test_suppliers.py
• **FR-007 & FR-013 (Purchase Orders & Partial Delivery):** IT_08, tests/test_purchase_orders.py
• **FR-008 (Product Search & Filtering):** tests/test_products.py, tests/test_main_routes_coverage.py
• **FR-009 & FR-014 (Sales, Stock, Revenue Reports & CSV/PDF Exports):** ST_02, ST_04, tests/test_sales_report.py, tests/test_stock_report.py, tests/test_revenue_report.py, tests/test_report_exports.py
• **FR-010 & NFR-010 (Audit Logging & 6-Month Retention):** IT_06, tests/test_products.py
• **FR-011 (Returns & Refunds):** IT_11, tests/test_sales.py
• **FR-012 & FR-015 (Password Reset & User Account Lifecycle):** tests/test_auth.py, tests/test_password_reset_email.py
• **NFR-005 (Atomic Rollback) & NFR-009 (Daily SQLite Backups):** IT_10, IT_12, tests/test_backup_database.py

13.2 Formal Test Cases Specification & Execution Results (UT_01–ST_04)

Test Case ID	Name of Module	Test Case Description	Preconditions	Test Steps	Test Data	Expected Results	Actual Result	Test Result
UT_01	Stock Management	To test stock deduction in the sales service after a sale	Product 'Wireless Router' exists in DB with stock = 10	1. Call create_sale() with product_id and quantity = 4 2. Query updated product stock in DB	Product: Wireless Router Price: 69.99 Initial stock: 10 Qty sold: 4	Sale record is returned and product stock updates from 10 to 6	create_sale() completed the sale and product stock decreased from 10 to 6.	PASS
UT_02	Sales Validation	Reject sale exceeding available stock	Product 'Mousepad' exists in DB with available stock = 4	1. Call create_sale() requesting 5 units when available stock is 4 2. Verify exception and inspect DB	Product: Mousepad Available stock: 4 Requested: 5	SaleValidationError ('Insufficient stock...') is raised; sale is rejected and stock remains 4	Overselling was rejected with SaleValidationError; no sale was created and stock remained 4.	PASS
IT_01	User Authentication	To test login functionality with valid credentials	Active user account (test-staff) exists in DB	1. Navigate to login page (/login) 2. Enter valid username and password 3. Submit login (POST /login)	Username: test-staff Password: StaffPass123	User is authenticated, session is established, and user is redirected to dashboard (/)	Manual: Redirected to dashboard (/). Automated: POST /login returned HTTP 200 and established session.	PASS
IT_02	User Authentication	To test login with invalid password	User account (deshna-test) exists in DB	1. Navigate to login page (/login) 2. Enter valid username and wrong password 3. Submit login (POST /login)	Username: deshna-test Password: WrongPassword123	Error message: 'Invalid username or password, or account deactivated.'	Login was rejected and 'Invalid username or password, or account deactivated.' was displayed.	PASS
IT_03	Product Management	To test adding a new product with valid data	Create Product form (/products/create) is accessible; SKU SPK-BT-100 is unused	1. Navigate to Create Product (/products/create) 2. Enter product details 3. Click Create Product	Name: Bluetooth Speaker SKU: SPK-BT-100 Price: 39.99 Qty: 20	Product is saved in DB and flash message 'Product created successfully.' is displayed	Product was saved in DB with SKU SPK-BT-100, price 39.99, stock 20, and confirmation message was displayed.	PASS

Test Case ID	Name of Module	Test Case Description	Preconditions	Test Steps	Test Data	Expected Results	Actual Result	Test Result
IT_04	Product Management	To test adding a product with duplicate SKU	Product 'First Item' with SKU UNIQUE-SKU already exists in DB	1. Navigate to Create Product (/products/create) 2. Enter existing SKU UNIQUE-SKU 3. Click Create Product	Name: Second Item SKU: UNIQUE-SKU (duplicate) Price: 20.00 Qty: 3	Error message: 'A product with this SKU already exists.' and duplicate is not saved	Duplicate SKU was rejected with 'A product with this SKU already exists.'; no duplicate product was created.	PASS
IT_05	Input Validation	Verify clear validation for invalid product or sale input	Create Product (/products/create) and Create Sale (/sales) forms are accessible	1. Submit blank product name or zero sale quantity 2. Inspect response and DB records	Product name: ' ' (blank) Sale quantity: 0	Clear validation message is displayed ('Product name is required.' / positive quantity); invalid record is not saved	Invalid input was rejected with a validation message and no invalid product or sale record was saved.	PASS
IT_06	Audit Logging	Verify product or stock changes create an audit entry	Authorized user (test-admin) is logged in; product 'Audit Mouse' exists with stock = 5	1. Update product stock from 5 to 10 via POST /products/<id>/edit 2. Query AuditLog model/table in DB	Product: Audit Mouse Stock update: 5 to 10	AuditLog DB record stores action='stock_edit', actor user_id, timestamp, and details '5 to 10'	Stock update created an AuditLog record in DB with the authenticated actor user_id and 'Stock changed from 5 to 10'.	PASS
IT_07	Billing + Inventory Integration	To test invoice generation and stock synchronization together	Product 'Keyboard' exists in DB with price = 25.50 and stock = 5	1. Submit sale for 2 units (POST /sales) 2. Follow redirect to invoice (/sales/<id>/invoice) 3. Verify invoice and DB stock	Product: Keyboard Price: 25.50 Initial stock: 5 Qty: 2	Invoice is generated ('Sales Invoice', subtotal 51.00) and stock is reduced by 2 (from 5 to 3) simultaneously	Invoice was generated and displayed, and inventory stock in DB was reduced from 5 to 3.	PASS
IT_08	Purchase Order + Inventory Integration	To test stock update when a Purchase Order is marked as Received	Manager or Admin logged in; Pending PO (PO-0001) exists for 'Desk Mat' (stock = 14) with quantity 20	1. Open Purchase Orders 2. Mark PO-0001 as Received (POST /purchase-orders/<id>/receive)	PO ID: PO-0001 Product: Desk Mat Initial stock: 14 Qty: 20	PO status becomes 'Received' and product stock increases by 20 (from 14 to 34)	Purchase Order PO-0001 receipt completed and the received quantity (20) increased stock from 14 to 34.	PASS
IT_09	Authentication + Role Access Integration	To test restricted access enforcement across modules	Staff account (test-staff) is logged in	1. Log in as Staff (test-staff) 2. Attempt to access Supplier module (GET /suppliers)	Username: test-staff Role: staff Route: /suppliers	Access denied with HTTP 403 Forbidden response.	Staff access to the restricted Supplier module (/suppliers) was denied (HTTP 403).	PASS
IT_10	Sales Transaction	Verify rollback of a failed multi-item sale	Multiple products exist in DB (Keyboard stock = 5, Monitor stock = 0)	1. Submit a multi-item sale where one item exceeds available stock 2. Inspect Sale, SaleItem, and Product records	Item 1: Keyboard (stock 5, qty 2) Item 2: Monitor (stock 0, qty 1)	No partial update; stock remains unchanged (5 and 0) and no incomplete sale is committed	Failed multi-item sale was rolled back; stock and sale records were not partially updated.	PASS
IT_11	Returns / Refunds	Verify return processing updates stock and records refund	Completed sale exists for 'USB-C Hub' (unit price 25.00, sold 3, stock = 7); return quantity is valid	1. Return 1 unit from the completed sale item 2. Inspect product stock and ProductReturn record	Product: USB-C Hub Unit price: 25.00 Return quantity: 1	Stock increases by 1 (from 7 to 8) and ProductReturn records quantity = 1 and refund_amount = 25.00	Valid return was processed; stock increased to 8 and ProductReturn recorded refund_amount 25.00.	PASS

Test Case ID	Name of Module	Test Case Description	Preconditions	Test Steps	Test Data	Expected Results	Actual Result	Test Result
IT_12	Database Backup	Verify successful database backup generation	SQLite database contains known test record ('test record'); BACKUP_DIR is configured	1. Run create_backup() in backup_database 2. Check generated backup file 3. Verify contents and PRAGMA integrity_check	Temporary SQLite DB with table backup_test containing ('test record',)	Backup file is created, contains the source record ('test record',), and passes SQLite integrity check ('ok')	Backup file was created; the known record was present and SQLite integrity check returned ok.	PASS
ST_01	Low Stock Alert System	To test alert generation when stock crosses the threshold	Product 'Wireless Earphones' has reorder threshold = 8 and initial stock = 10	1. Reduce stock below 8 through a sale of 3 units (10 to 7) 2. Check dashboard (GET /) for low-stock alert	Product: Wireless Earphones Threshold: 8 Stock reduced from 10 to 7	Low-stock alert banner ('Low-Stock Alert') is displayed on the dashboard for 'Wireless Earphones'	Low-stock alert was displayed on the dashboard when stock fell from 10 to 7 (below threshold 8).	PASS
ST_02	Reporting Module	To test sales report generation for a selected date range	Manager or Admin logged in; sales transaction ('USB Keyboard', 60.00) exists in system	1. Navigate to Sales Report (/reports/sales) 2. Select start and end range 3. Click Generate Report	Selected report range covering the recorded sale transaction	Sales report is generated and displays matching transactions ('USB Keyboard') and calculated total (60.00) for the selected range	Sales report displayed the matching sale details ('USB Keyboard') and calculated total (60.00).	PASS
ST_03	Role Access	Verify Staff, Manager and Admin permissions	Test accounts for Staff (test-staff), Manager (test-manager), and Admin (test-admin) exist	1. Sign in using each role 2. Attempt permitted and restricted actions across modules	Staff, Manager, and Admin test accounts	Each role accesses permitted modules (HTTP 200) and is denied restricted actions (HTTP 403)	Role-based access checks passed; restricted actions were denied (HTTP 403) for unauthorized roles.	PASS
ST_04	Stock Reporting	Verify stock movement report calculations	Manager or Admin logged in; product 'Report Mouse' (stock = 16) has sales, restocks, and returns in period	1. Open Stock Report (/reports/stock) 2. Select report period 3. Generate report and verify movement totals	Selected report period; Sold: 2, Restocked: 5, Returned: 1, Current stock: 16	Report shows Units Sold (2), Restocked (5), Returned (1), Net Movement (4), and Current Stock (16)	Stock report displayed the movement categories (2, 5, 1), calculated net movement (4), and current stock (16) correctly.	PASS

14. Test Metrics & Reporting

Metric	Target	Actual Measured Result
Planned Formal Test Cases Executed	100% (18 / 18)	100% (18 / 18 Executed — UT_01–UT_02, IT_01–IT_12, ST_01–ST_04)
Planned Formal Test Cases Passed	100% (18 / 18)	100% (18 Passed, 0 Failed)
Automated Pytest Suite Execution	100% Pass Rate	373 Executed, 373 Passed, 0 Failed, 0 Errors, 0 Skipped (across 20 test modules)
Statement / Line Coverage (app/ & scripts/)	≥ 90.00%	97.70% (1,950 / 1,996 statements covered; 46 missed across 28 files)
Branch Coverage (app/ & scripts/)	≥ 85.00%	91.50% (635 / 694 branches covered; 59 missed across 28 files)
Open Critical or High Defects	0	0 Open Defects

15. Approvals

Role	Name / Team	Approval Status
QA Lead	Karnati Uday Keerthan Reddy, K B Balavideshna, Kolluri Sruthi	Approved
Development Lead	Karnati Uday Keerthan Reddy, K B Balavideshna, Kolluri Sruthi	Approved
Product Owner / Course Instructor	Shravya S Madhusudan	Approved